

AI Engineering Roadmap

☰ Tags

Complete AI Engineering Roadmap: From Zero to Master

How to Use This Roadmap

This is a **progressive learning path** — follow it in order. Don't skip phases. Each phase builds on the previous one. Estimated total time: **6–12 months** depending on your pace.

PHASE 0 — Prerequisites (Weeks 1–3)

Before touching AI, make sure you have these foundations solid.

Mathematics (Don't Panic — You Need the Basics)

- **Linear Algebra** — Vectors, matrices, dot products, eigenvalues. These are the backbone of neural networks.
- **Calculus** — Derivatives, gradients, chain rule. This is how neural networks learn (backpropagation).
- **Probability & Statistics** — Bayes theorem, distributions, mean/variance. LLMs are probabilistic systems.
- **Resources:** 3Blue1Brown's *Essence of Linear Algebra* + *Essence of Calculus* (YouTube — free, visual, excellent)

Programming

- **Python** — Non-negotiable. Master: list comprehensions, decorators, async/await, OOP, type hints.
- **Git & GitHub** — Version control is mandatory in every AI project.

- **Linux/Terminal basics** — Most AI infrastructure runs on Linux.
- **Tools:** VS Code or Cursor IDE, Jupyter Notebooks, Google Colab (free GPU)

⚠ **What NOT to Do in Phase 0**

- Don't spend months perfecting math before writing code. Learn math *alongside* code.
 - Don't use outdated Python 2 tutorials.
 - Don't skip Git — you'll regret it later.
-

📌 **PHASE 1 — Machine Learning Fundamentals** (Weeks 4–8)

Core Concepts to Master

- **Supervised vs Unsupervised vs Reinforcement Learning** — understand the paradigms deeply.
- **Classic ML Algorithms** — Linear Regression, Logistic Regression, Decision Trees, Random Forest, SVM, K-Means, PCA.
- **The ML Pipeline** — Data collection → Preprocessing → Feature Engineering → Training → Evaluation → Deployment.
- **Overfitting, Underfitting, Bias-Variance Tradeoff** — master these concepts or you'll build broken models forever.
- **Evaluation Metrics** — Accuracy, Precision, Recall, F1, ROC-AUC, RMSE. Know when to use which.

Tools

- **Scikit-learn** — the standard ML library. Learn it deeply.
- **Pandas + NumPy** — data manipulation. Essential.
- **Matplotlib + Seaborn** — data visualization.
- **Kaggle** — practice with real datasets and competitions.

Best Learning Resources

- *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* by Aurélien Géron (Book — best ML book ever)
- Andrew Ng's ML Specialization on Coursera (free to audit)

⚠️ What NOT to Do

- Don't memorize algorithms — understand the *intuition* behind them.
 - Don't skip EDA (Exploratory Data Analysis) — it's how real engineers work.
 - Don't ignore data preprocessing. 80% of real ML work is cleaning data.
-

PHASE 2 — Deep Learning & Neural Networks (Weeks 9–14)

Core Concepts

- **Perceptrons → Multi-layer Neural Networks** — how they work mathematically.
- **Backpropagation & Gradient Descent** — understand this deeply. It's how every model learns.
- **Activation Functions** — ReLU, Sigmoid, Tanh, GELU, SiLU. Why they matter.
- **CNNs (Convolutional Neural Networks)** — for images and vision tasks.
- **RNNs, LSTMs, GRUs** — for sequences and time series (predecessor to Transformers).
- **Regularization** — Dropout, Batch Normalization, L1/L2.
- **Optimizers** — SGD, Adam, AdamW. Know the differences.

Tools

- **PyTorch** ★ — Learn this first. It's the industry standard for research and production AI in 2025.
- **TensorFlow/Keras** — still used in production; know the basics.
- **Weights & Biases (wandb)** — experiment tracking. Use it from day one.

- **Hugging Face** — the GitHub of AI models. Start getting familiar here.

⚠ **What NOT to Do**

- Don't start with TensorFlow 1.x tutorials — they're outdated.
 - Don't skip PyTorch. Almost every modern LLM repo uses PyTorch.
 - Don't train on CPU — use Google Colab or Kaggle free GPU tiers.
-

📌 **PHASE 3 — The Transformer Revolution & LLMs (Weeks 15–22)**

This is the **heart of modern AI**. Study this phase the most carefully.

Part A — Understanding Transformers

Start by reading (and re-reading) "**Attention Is All You Need**" (the original 2017 paper). Then understand:

- **Self-Attention & Multi-Head Attention** — the core mechanism. Understand it mathematically and intuitively.
- **Positional Encoding** — how Transformers understand word order.
- **Encoder vs Decoder vs Encoder-Decoder architectures** — BERT is encoder-only; GPT is decoder-only; T5 is both.
- **Tokenization** — BPE, WordPiece, SentencePiece. How text becomes numbers.
- **Embeddings** — Word2Vec → modern embeddings. Semantic meaning in vector space.
- **Context Window** — what it means, why it matters, how modern models extend it.
- **KV Cache** — critical for understanding LLM inference efficiency.

Best Resource: Andrej Karpathy's "*Let's build GPT from scratch*" (YouTube) — build a mini-GPT yourself. This is the single best tutorial in AI education.

Part B — Modern LLMs (Know Them Deeply)

Model Family	Creator	Key Strength
GPT-4o / o3 / o4-mini	OpenAI	Reasoning, multimodal
Claude 3.5 / Claude 4	Anthropic	Safety, long context, coding
Gemini 2.0 / 2.5 Pro	Google DeepMind	Multimodal, long context
Llama 3.3 / 4	Meta	Open-source, fine-tunable
Mistral / Mixtral	Mistral AI	Efficient, open-source
DeepSeek V3 / R1	DeepSeek	Reasoning, cost-efficient
Qwen 2.5	Alibaba	Multilingual, open-source

Study each model's architecture differences, training approach, and strengths. Read their technical reports.

Part C — How LLMs Are Trained

- **Pre-training** — next-token prediction on massive datasets (trillions of tokens).
- **Fine-tuning** — adapting a base model to specific tasks.
- **RLHF (Reinforcement Learning from Human Feedback)** — how ChatGPT learned to be helpful.
- **RLAIF** — using AI feedback instead of human feedback (cheaper, scalable).
- **DPO (Direct Preference Optimization)** — modern alternative to RLHF. Simpler, widely used.
- **Constitutional AI** — Anthropic's method for safe alignment.
- **Scaling Laws** — Chinchilla laws. Why model size + data size matter together.

Part D — Key LLM Concepts for Engineers

- **Temperature, Top-P, Top-K** — sampling parameters. Know how to tune them.
- **System prompts vs User prompts** — how instruction tuning works.
- **Hallucination** — what causes it, how to mitigate it (RAG, grounding, verification).
- **Context window limits** — how to handle them in real applications.
- **Quantization** — INT8, INT4, GGUF format. Run large models on smaller hardware.

- **LoRA / QLoRA** — parameter-efficient fine-tuning. The standard way to fine-tune in 2025.

What NOT to Do

- Don't just use ChatGPT — build with the APIs and understand what's happening underneath.
 - Don't skip reading papers. Get comfortable with arxiv.org — it's where all new AI knowledge is published first.
 - Don't treat LLMs as magic black boxes. Understand the mechanism.
-

PHASE 4 — Prompt Engineering & LLM APIs (Weeks 23–26)

Prompt Engineering (A Real Skill, Not a Gimmick)

- **Zero-shot vs Few-shot prompting** — when to use each.
- **Chain-of-Thought (CoT)** — "Let's think step by step." Dramatically improves reasoning.
- **Tree-of-Thought** — exploring multiple reasoning paths.
- **ReAct prompting** — Reasoning + Acting pattern (foundation of agents).
- **Role prompting, Persona prompting**
- **Structured output prompting** — forcing JSON, XML outputs reliably.
- **Prompt injection & security** — know the attacks and defenses.

Working With LLM APIs

- **OpenAI API** — GPT-4o, o3, structured outputs, function calling, vision.
- **Anthropic API** — Claude models, tool use, long context (200k tokens).
- **Google Gemini API** — multimodal, 1M context window.
- **Together AI / Groq / Fireworks** — fast inference for open-source models.
- **Ollama** — run open-source LLMs locally (Llama, Mistral, etc.)

Tools

- **LiteLLM** — unified interface to call any LLM with one API. Extremely useful.
 - **Instructor** — reliable structured outputs from any LLM.
 - **DSPy** — programmatic prompting, auto-optimize prompts. Next-generation approach.
-

PHASE 5 — RAG (Retrieval Augmented Generation) (Weeks 27–30)

RAG is the most important production AI pattern in 2025. Master it completely.

What RAG Solves

LLMs hallucinate and have knowledge cutoffs. RAG connects them to real, up-to-date knowledge bases.

RAG Pipeline (Learn Each Step)

1. **Document Loading & Parsing** — PDFs, Word docs, web pages, databases. Tools: LangChain document loaders, Unstructured.io, LlamaParse.
2. **Chunking Strategies** — Fixed-size, sentence-based, semantic chunking, recursive character splitting. Wrong chunking = bad RAG. This matters a lot.
3. **Embedding Models** — Convert text chunks to vectors. Top models: `text-embedding-3-large` (OpenAI), `voyage-3` (Voyage AI), `nomie-embed-text` (open-source).
4. **Vector Databases** — Store and search embeddings. Know these:
 - **Pinecone** — managed, production-ready
 - **Weaviate** — open-source, powerful
 - **Qdrant** — fast, Rust-based, excellent
 - **ChromaDB** — great for local development
 - **PGVector** — if you're already on PostgreSQL
5. **Retrieval** — Similarity search (cosine, dot product), Hybrid search (vector + keyword with BM25), MMR (Maximal Marginal Relevance) for diversity.

6. Reranking — After retrieval, rerank results for better relevance. Tools: Cohere Reranker, FlashRank, BGE Reranker.

7. Generation — Feed retrieved context + query to LLM. Generate answer.

Advanced RAG Techniques

- **HyDE (Hypothetical Document Embeddings)** — generate hypothetical answer, embed it, search with that.
- **Self-RAG** — model decides when to retrieve vs answer from memory.
- **RAPTOR** — recursive summarization for hierarchical retrieval.
- **GraphRAG** — Microsoft's approach using knowledge graphs. Very powerful for complex queries.
- **Agentic RAG** — agents that iteratively search and refine.

RAG Evaluation (Critical — Don't Skip)

- **RAGAs framework** — evaluate Faithfulness, Answer Relevancy, Context Precision, Context Recall.
 - **TruLens** — alternative RAG evaluation framework.
 - **ARES** — automated RAG evaluation system.
-

PHASE 6 — AI Agents (Weeks 31–38)

This is the **cutting edge** of AI engineering. Agents are LLMs that can take actions, use tools, and complete multi-step tasks autonomously.

Core Agent Concepts

- **ReAct Pattern** — Reason → Act → Observe → Repeat. The foundation of all agents.
- **Tool Use / Function Calling** — giving LLMs the ability to call external functions (APIs, databases, code execution).
- **Memory Systems:**
 - *In-context memory* — within the prompt window

- *External memory* — databases, vector stores
- *Episodic memory* — remembering past conversations/experiences
- *Semantic memory* — facts and knowledge
- **Planning** — task decomposition, sub-goal generation.
- **Multi-Agent Systems** — multiple specialized agents collaborating.

Agent Architecture Patterns

- **Single Agent with Tools** — simplest, good for focused tasks.
- **Supervisor + Worker Agents** — orchestrator delegates to specialized sub-agents.
- **Peer-to-Peer Multi-Agent** — agents communicate horizontally.
- **Human-in-the-Loop** — agent pauses for human approval on critical steps.
Essential for production.

Top Agent Frameworks in 2025

LangChain — the original, massive ecosystem, great for RAG + agents. Can be over-engineered. Use LangChain Expression Language (LCEL).

LangGraph ★ — built on LangChain, graph-based agent orchestration. Best for complex, stateful, multi-agent workflows. The current gold standard for production agents.

LlamaIndex — excellent for data-heavy applications, RAG, and structured data agents.

CrewAI ★ — role-based multi-agent framework. Easy to get started, great for collaborative agent teams.

AutoGen (Microsoft) — conversational multi-agent framework. Strong for coding and research agents.

Pydantic AI — type-safe agent framework. Clean, production-grade.

Smolagents (Hugging Face) — lightweight, code-first agents. Very clean design.

OpenAI Agents SDK — OpenAI's official agent framework. Simple, powerful if you're using GPT models.

Agent Tools to Master

- **Code Execution** — agents that write and run code (E2B sandbox, Docker).
- **Web Search** — Tavily Search API, SerpAPI, Brave Search API.
- **Browser Control** — Playwright, Selenium, Browserbase, Stagehand.
- **File Operations** — read/write documents, parse PDFs.
- **API Calls** — agents connecting to any REST API.
- **Computer Use** — Claude's computer use, OpenAI's Operator. Agents controlling a full computer.

MCP (Model Context Protocol) ★ NEW & CRITICAL

Anthropic's **Model Context Protocol** is becoming the standard way to connect AI agents to tools and data sources. Think of it as "USB for AI." Learn it — it's rapidly becoming the industry standard. Every major AI company is adopting it.

Agentic Design Principles

- **Start simple** — begin with single-agent + tools before building multi-agent systems.
- **Determinism first** — make your agent as predictable as possible before adding autonomy.
- **Observability** — log every LLM call, tool call, and decision. You can't debug what you can't see.
- **Graceful failure** — agents must handle tool failures, rate limits, and hallucinations.
- **Human checkpoints** — always include human approval for irreversible actions (sending emails, making payments, deleting data).

⚠ What NOT to Do with Agents

- Don't build complex multi-agent systems before mastering single agents.
- Don't give agents access to irreversible actions (production databases, real payments) without human-in-the-loop.
- Don't forget to set token budgets and iteration limits — agents can loop forever.
- Don't ignore security — prompt injection attacks can hijack agents.

PHASE 7 — Fine-Tuning & Model Customization (Weeks 39–44)

When to Fine-Tune (vs RAG vs Prompting)

Fine-tune when you need: consistent output format/style, specialized domain knowledge baked in, faster inference, or behavior that prompting alone can't achieve.

Don't fine-tune when: RAG would solve the problem, you have limited data (< 100 examples), or the cost isn't justified.

Fine-Tuning Techniques

- **Full Fine-tuning** — update all weights. Expensive. Only for massive infrastructure.
- **LoRA (Low-Rank Adaptation)** ★ — the standard. Add small trainable matrices. ~same performance as full fine-tuning at a fraction of cost.
- **QLoRA** — LoRA + quantization. Fine-tune a 70B model on a single GPU. Game-changing.
- **PEFT (Parameter-Efficient Fine-Tuning)** — umbrella term. Hugging Face library.
- **Instruction Fine-tuning** — teach the model to follow instructions.
- **DPO Fine-tuning** — align model to preferred responses without reinforcement learning complexity.

Fine-Tuning Tools

- **Hugging Face Transformers + TRL** — the standard library stack.
- **Unsloth** ★ — 2–5x faster fine-tuning with 80% less VRAM. Incredible tool.
- **Axolotl** — flexible fine-tuning config-based framework.
- **LLaMA-Factory** — easy UI-based fine-tuning for Llama models.
- **Modal / Lambda Labs / RunPod** — affordable GPU cloud for fine-tuning.
- **OpenAI Fine-tuning API** — easiest path if you want to fine-tune GPT-4o-mini.

Evaluation After Fine-Tuning

- **Eleuther AI LM Evaluation Harness** — standard benchmark suite.

- **MT-Bench** — multi-turn conversation benchmarks.
 - **Custom evals** — always build task-specific evaluations for your use case.
-

PHASE 8 — Production AI Engineering (Weeks 45–52)

This is where most tutorials stop, but where real engineering begins.

LLMOps — Operating LLMs in Production

- **LangSmith** ★ — LLM observability, tracing, debugging, evaluation. Works with LangChain and standalone. Essential.
- **Langfuse** — open-source alternative to LangSmith.
- **Helicone** — LLM proxy with analytics, caching, rate limiting.
- **Arize Phoenix** — ML + LLM observability platform.
- **Portkey** — AI gateway with routing, fallbacks, cost tracking.

Deployment & Serving

- **FastAPI** — build REST APIs around your AI systems. The standard.
- **vLLM** ★ — fastest open-source LLM serving. PagedAttention = massive throughput gains.
- **TGI (Text Generation Inference)** — Hugging Face's production serving framework.
- **Triton Inference Server** — NVIDIA's high-performance serving platform.
- **BentoML** — package and deploy ML models easily.

Infrastructure

- **Docker + Kubernetes** — containerize and orchestrate AI services.
- **AWS Bedrock / Azure AI Studio / Google Vertex AI** — managed LLM platforms in cloud.
- **Modal** — serverless GPU compute. Excellent for AI workloads.
- **Replicate** — deploy and share ML models easily.

- **Hugging Face Inference Endpoints** — one-click deployment for HF models.

Cost Optimization (Critical in Production)

- **Caching** — semantic caching with GPTCache or Redis. Cache LLM responses to save cost.
- **Model routing** — use cheap models (GPT-4o-mini, Haiku) for simple queries, expensive models for complex ones.
- **Prompt compression** — LLMingua. Compress prompts by 3–5x without losing quality.
- **Streaming** — always stream responses for better UX.
- **Batching** — batch API requests when latency tolerance is high.

Security for AI Systems

- **Prompt injection prevention** — validate and sanitize inputs.
- **Output validation** — use Guardrails AI or Nemo Guardrails.
- **PII detection** — detect and mask personal information before sending to LLMs.
- **Rate limiting** — protect your API from abuse.

Evaluation in Production

- **A/B testing** — test prompt changes against each other with real traffic.
- **LLM-as-judge** — use a strong LLM to evaluate outputs at scale.
- **Human feedback loops** — collect thumbs up/down, use to improve.
- **Evals first** — build your evaluation suite *before* building the system. This is the mark of a senior AI engineer.

PHASE 9 — Advanced & Cutting-Edge Topics (Ongoing)

These are frontier topics. Follow alongside the rest of your learning.

- **Multimodal AI** — vision + language (GPT-4o, Gemini, LLaVA, Qwen-VL). Image, audio, video understanding.
- **Long Context** — working with 1M+ token contexts (Gemini 2.5 Pro). Needle-in-haystack evaluation.
- **Reasoning Models** — o3, DeepSeek-R1, Qwen QwQ. Chain-of-thought inference-time compute.
- **Voice AI** — real-time voice agents with Whisper, ElevenLabs, Cartesia, Deepgram.
- **AI for Code** — GitHub Copilot internals, Cursor, Aider, Claude Code architecture.
- **Mixture of Experts (MoE)** — how Mixtral and GPT-4 scale efficiently.
- **Synthetic Data Generation** — use LLMs to generate training data. The future of ML.
- **Knowledge Graphs + LLMs** — Neo4j + LLMs for structured reasoning.

Master Stack Summary (2025)

Category	Tool
Language	Python 3.11+
ML Framework	PyTorch
LLM Frameworks	LangChain + LangGraph, LlamaIndex
Agent Frameworks	CrewAI, LangGraph, Pydantic AI
LLM APIs	OpenAI, Anthropic, Google Gemini
Local LLMs	Ollama + Open WebUI
Vector DB	Qdrant (production), ChromaDB (dev)
Embeddings	text-embedding-3-large, voyage-3
Fine-tuning	Unsloth + TRL + LoRA/QLoRA
Observability	LangSmith + Langfuse
Serving	vLLM + FastAPI
Deployment	Docker + Modal / Replicate
Evaluation	RAGAs, LLM-as-judge, custom evals

Category	Tool
Experiment Tracking	Weights & Biases

Essential Papers to Read

1. *Attention Is All You Need* (2017) — the Transformer paper
2. *BERT* (2018) — encoder-only Transformers
3. *GPT-3* (2020) — few-shot learning from scale
4. *InstructGPT* (2022) — RLHF explained
5. *Chain-of-Thought Prompting* (2022) — Wei et al.
6. *ReAct: Reasoning + Acting* (2022) — foundation of agents
7. *Toolformer* (2023) — LLMs learning to use tools
8. *LLaMA* papers (2023–2024) — open-source LLMs
9. *DeepSeek-R1* (2025) — reasoning through RL
10. *Scaling Laws* (Kaplan et al.) — how to scale LLMs efficiently

Global "What NOT to Do" List

- **Don't** chase every new model release. Go deep on fundamentals instead.
- **Don't** build without evals. If you can't measure it, you can't improve it.
- **Don't** over-engineer with frameworks early. Understand the primitives first, then abstract.
- **Don't** ignore security and safety in production AI.
- **Don't** learn in isolation. Join communities: Hugging Face Discord, r/MachineLearning, r/LocalLLaMA, AI Twitter/X.
- **Don't** skip building projects. Reading alone does nothing.
- **Don't** wait until you "know enough" to start building. Start building on day one.

Projects to Build (Prove Your Skills)

1. **Phase 3** — Build a mini-GPT from scratch following Karpathy's tutorial
 2. **Phase 4** — Build a CLI chatbot using OpenAI + Anthropic APIs
 3. **Phase 5** — Build a RAG system over your own documents (PDF, Notion, etc.)
 4. **Phase 6** — Build a research agent that searches the web and summarizes findings
 5. **Phase 6** — Build a multi-agent coding assistant
 6. **Phase 7** — Fine-tune a Llama model on a custom dataset
 7. **Phase 8** — Deploy a production RAG API with LangSmith observability
 8. **Capstone** — Build a full AI product: agent + RAG + fine-tuned model + production deployment + evaluation suite
-

Communities & Staying Current

- **Twitter/X:** Follow @karpathy, @sama, @ylecun, @dair_ai, @_philschmid, @hwchase17
 - **Hugging Face Discord** — most active open-source AI community
 - **r/LocalLLaMA** — open-source LLM discussions
 - **Papers with Code** — track SOTA and find implementations
 - **The Batch (deeplearning.ai newsletter)** — weekly AI news
 - **Ahead of AI newsletter** — deep technical breakdowns by Sebastian Raschka
-

Final Mentor Advice: The biggest mistake people make is tutorial hell — watching and reading without building. After every phase, build something. Break it. Fix it. That's where real learning happens. The AI field moves fast, but the fundamentals — attention, embeddings, reasoning, retrieval — remain stable. Master those, and you'll adapt to whatever comes next. Now go build something. 🚀